

মডিউল নোট: Classification এবং Logistic Regression

মেশিন লার্নিং অ্যান্ড ডাটা সায়েন্স ফাউন্ডেশন

এই মডিউলটি মেশিন লার্নিংয়ের অন্যতম গুরুত্বপূর্ণ ভিত্তি। এখানে আমরা শিখব কীভাবে ডেটাকে বিভিন্ন ক্যাটাগরিতে ভাগ করা যায় এবং তার পেছনের গাণিতিক ও প্রাঙ্কিক্যাল বিষয়গুলো কীভাবে কাজ করে।

1. Introduction to Classification Problem

মেশিন লার্নিংয়ে যখন আমাদের টার্গেট ভ্যারিয়েবল বা আউটপুটটি কোনো নির্দিষ্ট সংখ্যা না হয়ে একটি **Category** বা **Class** হয়, তখন তাকে **Classification Problem** বলে।

- **Binary Classification:** একটি ইমেইল 'Spam' নাকি 'Not Spam'? অথবা একটি টিউমার 'Malignant' (ক্ষতিকর) নাকি 'Benign' (ক্ষতিকর নয়)?
- **Multiclass Classification:** একটি ছবির পশুটি 'বিড়াল', 'কুকুর' নাকি 'পাখি'?

Regression বনাম Classification-এর পার্থক্য

বৈশিষ্ট্য	Regression (রিগ্রেশন)	Classification (ক্লাসিফিকেশন)
আউটপুট টাইপ	Continuous (যেকোনো সংখ্যা বা মান)	Discrete / Categorical (নির্দিষ্ট ক্লাস)
উদাহরণ	বাড়ির দাম নির্ধারণ, আগামীকালের তাপমাত্রা	ইমেইল স্প্যাম ডিটেকশন, পাস/ফেল নির্ধারণ
প্রধান অ্যালগরিদম	Linear Regression, Polynomial Regression	Logistic Regression, Decision Tree, SVM

❌ Classification-এর জন্য Linear Regression কেন উপযুক্ত নয়?

১. আউটপুট সীমার বাইরে চলে যাওয়া (>1 বা <0): Linear Regression-এর আউটপুট রেঞ্জ হলো $-\infty$ থেকে $+\infty$ । কিন্তু Binary Classification-এ আমাদের আউটপুট দরকার 0 অথবা 1 । Linear Regression ব্যবহার করলে প্রেডিকশন 1.5 বা -0.2 চলে আসতে পারে, যার কোনো বাস্তব অর্থ হয় না।

২. **Outliers-এর প্রতি অতিমাত্রায় সংবেদনশীল:** ডেটাসেটে যদি কোনো চরম মান (Outlier) চলে আসে, তবে Linear Regression-এর Decision Boundary মারাত্মকভাবে স্থানচ্যুত হয়। ফলে সঠিক ডেটাও ভুল প্রেডিক্ট হতে পারে।

2. Core Concepts of Logistic Regression

Logistic Regression নামটির সাথে "Regression" যুক্ত থাকলেও, এটি আসলে একটি **Classification Algorithm**। এটি মূলত কোনো ঘটনা ঘটার সম্ভাবনা (Probability) পরিমাপ করে।

Sigmoid Function (সিগময়েড ফাংশন)

Linear Regression-এর সরলরেখাকে একটি নির্দিষ্ট সীমার (0 থেকে 1) মধ্যে নিয়ে আসার জাদুকরী টুলটিই হলো **Sigmoid Function**। এর সমীকরণটি হলো:

$$g(z) = 1 / (1 + e^{-z})$$

এখানে $z = mx + c$ (বা $w^T x + b$)। z -এর মান যতই বড় বা ছোট হোক না কেন, $g(z)$ -এর মান সবসময় 0 থেকে 1-এর মধ্যে থাকবে। এর গ্রাফটি দেখতে ইংরেজি 'S' অক্ষরের মতো হয়।

Data Transformation ও Probability-based Prediction

- **Transformation:** প্রথমে ফিচারগুলোকে লিনিয়ার ইকুয়েশনে (z) বসানো হয়, তারপর সেই z -এর মানকে Sigmoid ফাংশনের মধ্য দিয়ে পাস করিয়ে একটি **Probability (সম্ভাবনা)** পাওয়া যায়।
- **Decision Threshold:** সাধারণত স্ট্যান্ডার্ড থ্রেশহোল্ড ধরা হয় 0.5।
 - যদি Probability ≥ 0.5 হয় $\rightarrow$ আউটপুট বা ক্লাস হবে 1 (Positive)
 - যদি Probability < 0.5 হয় $\rightarrow$ আউটপুট বা ক্লাস হবে 0 (Negative)

3. Cost Function & Model Training

মডেল কতটা ভালো বা খারাপ কাজ করছে তা পরিমাপ করার জন্য আমাদের একটি Cost Function দরকার। Linear Regression-এর Mean Squared Error (MSE) এখানে ব্যবহার করা যায় না, কারণ সিগময়েড ফাংশনের কারণে MSE ব্যবহার করলে গ্রাফটি Non-convex হয়ে যায়, যা অপ্টিমাইজ করা কঠিন।

Logistic Regression Cost Function (Log Loss)

Logistic Regression-এ **Binary Cross-Entropy** বা **Log Loss** ফাংশন ব্যবহার করা হয়:

$$J(w, b) = - (1/m) \sum [y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))]]$$

সহজ ভাষায়, আসল উত্তর (y) যদি 1 হয় এবং মডেল যদি 1 প্রেডিক্ট করে, তবে Cost হবে 0। কিন্তু মডেল যদি 0 প্রেডিক্ট করে, তবে Cost বা পেনাল্টি হবে অনেক বেশি (কাছাকাছি ইনফিনিটি)। ঠিক একইভাবে $y = 0$ এর জন্যও এটি কাজ করে।

⚙️ Cost Optimization & Gradient Descent

মডেল ট্রেনিংয়ের উদ্দেশ্য হলো এই Cost (J)-এর মান যথাসম্ভব কমিয়ে আনা। এর জন্য আমরা ব্যবহার করি **Gradient Descent** অ্যালগরিদম।

- **Weights & Bias Initialize করা:** শুরুতে Random বা 0 দিয়ে Weight (w) ও Bias (b) ধরা হয়।
- **Gradient বের করা:** Cost Function-এর ডেরিভেটিভ (Derivative) বা ঢাল বের করা হয়।
- **Weight Update:** নিচের সূত্র অনুযায়ী প্রতি ইটারেশনে Weight ও Bias পরিবর্তন করা হয়:

$$w := w - \alpha * (\partial J / \partial w)$$

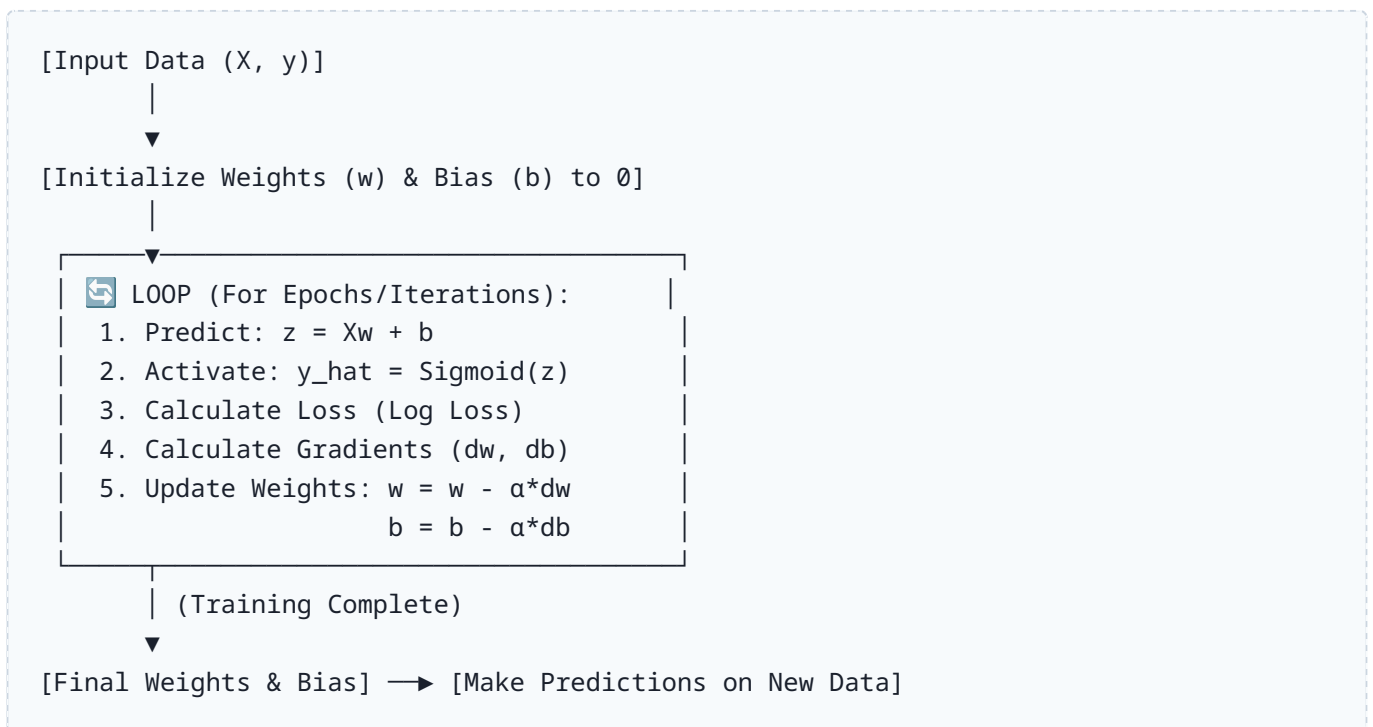
$$b := b - \alpha * (\partial J / \partial b)$$

(এখানে α হলো *Learning Rate*, যা নির্ধারণ করে মডেল কত দ্রুত শিখবে।)

4. Implementing Logistic Regression from Scratch

তত্ত্বীয় বিষয়গুলো ভালোভাবে বোঝার পর স্ক্র্যাচ (কোনো রেডিমেড লাইব্রেরি ছাড়া) থেকে কোড লেখার মাধ্যমে পুরো Workflow-টি সম্পন্ন হয়।

🔧 সম্পূর্ণ Workflow সংক্ষেপে:



💡 নোটের শেষ কথা: এই মডিউলটি সফলভাবে শেষ করার মাধ্যমে আপনি মেশিন লার্নিংয়ের **Classification** ডোমেইনে প্রবেশ করলেন। এই লজিকটিই পরবর্তীতে **Neural Network** এবং **Deep Learning**-এর ক্ষেত্রে "Activation Function" ও "Loss Optimization" হিসেবে ব্যাপকভাবে ব্যবহৃত হবে।